Web Mining Projects

1. Web Structure Mining

In this project, students are tasked with analyzing a social media network to identify key influencers and community structures using Python. The project will involve preprocessing and visualizing the network data, applying community detection algorithms, and interpreting results effectively. The grading will focus on their proficiency in using Python libraries such as NetworkX or igraph for network analysis. A recommended dataset for this project is Twitter API data, which includes information on follower relationships and interactions that can be used to construct a social network graph.

Project Details:

- **Objective:** Utilize Python to preprocess, analyze, and visualize a social media network to uncover influential users and distinct community structures within the graph.
- **Tasks:**
    - **Data Preprocessing:** Clean and structure raw data from social media platforms into a usable graph format. Students will need to handle data extraction, normalization, and transformation to prepare it for analysis.
    - **Network Analysis:** Apply graph theory concepts to identify central nodes (influencers) and clusters (communities) within the network. This may involve metrics like centrality, density, and modularity.
    - **Visualization:** Create visual representations of the network that highlight community structures, influential users, and other relevant insights using libraries like NetworkX or igraph.
    - **Interpretation:** Analyze the results to provide insights into the social structure of the network, discussing the roles of identified influencers and the characteristics of the communities.

Grading Criteria:

- **Proficiency in Network Analysis:** Accuracy and depth of network analysis, demonstrated proficiency in using Python libraries like NetworkX or igraph to handle complex network data.

- **Effectiveness of Data Preprocessing:** Clarity and efficiency in data preprocessing steps, ensuring that the social media data is correctly transformed into a graph format suitable for analysis.
- **Quality of Visualizations:** Creativity and clarity in the visual representation of the network, making complex data understandable and visually compelling.
- **Depth of Interpretation:** The ability to derive meaningful and actionable insights from the network analysis, explaining the implications of network metrics and community structures in the context of social media dynamics.

Recommended Dataset:

- **Twitter API Data:** This dataset includes follower relationships, user interactions, and other social media activities that can be modeled as a network. Students will access this data through the Twitter API, requiring them to handle API requests and data extraction as part of the project.

This project is designed to provide students with hands-on experience in web structure mining using real-world data, enhancing their skills in Python programming, network analysis, and data visualization. Through this project, students will gain valuable insights into the practical challenges and methodologies in social network analysis, preparing them for careers in data science and analytics.

2. Community Detection

This project will require students to implement and evaluate various community detection algorithms on a citation network to uncover significant research clusters. The implementation should be done in Python, utilizing libraries like NetworkX for graph operations. Students will be graded on their algorithm implementation, the clarity of community comparisons, and the depth of network analysis. The DBLP Citation Network from the SNAP dataset is suitable for this project.

Project Details:

- **Objective:** Use Python to apply and assess different community detection algorithms on a citation network, aiming to reveal key research clusters and understand the structure of academic collaborations.

- **Tasks:**

- **Data Handling:** Load and preprocess the DBLP Citation Network data to prepare it for analysis. This involves parsing the data to fit a graph-based model where nodes represent papers or authors and edges represent citations.
- **Algorithm Implementation:** Implement various community detection algorithms using Python libraries such as NetworkX. Algorithms might include modularity-based methods, hierarchical clustering, or density-based techniques.
- **Evaluation:** Compare and evaluate the effectiveness of different community detection algorithms. This could be based on metrics such as the modularity score, the size and coherence of detected communities, or the stability of the communities across different algorithm parameters.
- **Analysis and Reporting:** Analyze the identified communities to draw insights about the major research areas, collaboration patterns, and influential papers or authors within the field.

Grading Criteria:

- **Accuracy of Algorithm Implementation:** Correctness in implementing community detection algorithms, with attention to detail in following algorithmic steps as described in scholarly resources or documentation.
- **Clarity of Community Comparisons:** Ability to clearly compare and contrast the outcomes of different community detection algorithms, presenting the results in a manner that is understandable and informative.
- **Depth of Network Analysis:** Depth of the analysis provided on the community structures, including the interpretation of what these communities represent within the broader academic landscape.
- **Quality of Code and Documentation:** Organized, well-commented, and readable Python code along with comprehensive documentation that explains the methodology, findings, and significance of the analysis.

Recommended Dataset:

- **DBLP Citation Network:** Available from the Stanford Large Network Dataset Collection (SNAP), this dataset includes detailed records of citations among academic papers. It is a robust dataset for exploring community detection in a real-world academic setting.

This project aims to provide students with practical experience in the application of community detection algorithms within the realm of citation networks, leveraging Python's powerful libraries to handle complex graph-based data. The project not only enhances technical skills in network analysis but also fosters a deeper understanding of academic collaboration patterns, potentially influencing future research methodologies and strategies.

3. Overlapping Communities

Students will investigate overlapping community structures in a collaborative tagging system used on platforms like Stack Overflow. The grading will consider the innovative application of overlapping community detection methods, the effectiveness in identifying significant tags, and the quality of insights derived. The Stack Overflow Tags Dataset, which includes user-tag interactions, is recommended for this project.

Project Details:

- Objective: Investigate and analyze overlapping community structures in a collaborative tagging system, utilizing Python and libraries suited for handling complex graph data, such as NetworkX or community detection libraries that support overlapping community algorithms.

- **Tasks:**

    - **Data Preparation:** Import and preprocess the Stack Overflow Tags Dataset to create a graph where nodes represent users and tags, and edges represent the application of tags by users.
    - **Community Detection Implementation:** Implement overlapping community detection algorithms, such as Clique Percolation Method (CPM) or algorithms based on fuzzy clustering, to identify communities within the network that share common tags but also participate in multiple tag groups.
    - **Evaluation and Analysis:** Evaluate the effectiveness of the community detection methods based on how well they identify significant and meaningful tag-based communities. Analyze the structure and content of these communities to understand the overlap and the relationship between different areas of interest or expertise on Stack Overflow.
    - **Insights and Reporting:** Derive insights about community behavior, expertise distribution, and collaborative tagging practices. Prepare a comprehensive report

that not only details the methodologies and results but also discusses the implications of overlapping communities in collaborative environments.

Grading Criteria:

- **Innovation in Community Detection:** Creativity and innovation in applying or adapting overlapping community detection methods to the collaborative tagging system. Exploration of new or less traditional methods is encouraged.
- **Effectiveness in Identifying Significant Tags:** Accuracy and relevance in identifying significant tags that define the communities. This includes how well the detected communities correspond to known or expected patterns within Stack Overflow.
- **Quality of Insights Derived:** Depth of the insights gained from the analysis of overlapping communities. This involves understanding the multi-dimensional interaction of users with various tags and how these interactions define community boundaries.
- **Quality of Code and Documentation:** Clarity and organization of the Python code, use of appropriate libraries and tools, and thorough documentation that explains the process, findings, and implications of the community analysis.

Recommended Dataset:

- **Stack Overflow Tags Dataset:** This dataset includes interactions between users and tags on Stack Overflow, providing a rich source of data for analyzing how users engage with different programming topics and how these topics overlap.

This project is designed to give students hands-on experience with advanced community detection techniques in a real-world dataset. It aims to enhance their understanding of complex social structures within digital collaboration platforms and improve their skills in data processing, graph analysis, and Python programming.

4. Web Content Mining in Python: Sentiment Analysis on E-commerce Reviews

In this project, students will explore the realm of web content mining by conducting sentiment analysis on customer reviews from e-commerce platforms. The project will focus on using Python and its powerful libraries like NLTK and spaCy to process text and extract sentiment, aiming to gauge product satisfaction levels and derive meaningful insights from customer feedback.

Project Details:

- **Objective:** Implement sentiment analysis using Python to evaluate customer sentiments expressed in product reviews on e-commerce platforms. The goal is to determine overall satisfaction levels and identify key factors influencing customer opinions.

- **Tasks:**

    - **Data Preparation:** Load and preprocess data from the Amazon Product Reviews dataset to format suitable for sentiment analysis. This includes cleaning the text (removing noise, normalizing text), tokenizing, and possibly tagging parts of speech.
    - **Sentiment Analysis Implementation:** Utilize text processing libraries like NLTK or spaCy to implement sentiment analysis. This could involve building a sentiment classifier from scratch or using pre-trained sentiment models to assess the polarity of the reviews.
    - **Evaluation of Models:** Evaluate the accuracy of the sentiment classification models through metrics such as precision, recall, and F1-score. Analyze the performance of different models or approaches to determine which provides the most reliable and consistent results.
    - **Insight Extraction and Reporting:** Analyze the output of the sentiment analysis to extract insights about customer preferences, satisfaction levels, and common grievances or praises. Compile these findings into a report that discusses the implications for product improvement and customer satisfaction strategies.

Grading Criteria:

- **Accuracy of Sentiment Classification Models:** The precision and accuracy with which the sentiment analysis models classify the sentiment of the reviews.
- **Innovativeness in Methodology:** Creativity in applying text processing and sentiment analysis techniques. Utilization of advanced features in NLTK or spaCy, or integration of machine learning models could be considered here.
- **Depth of Insights Derived:** The depth and relevance of insights extracted from the sentiment analysis, including how these insights could potentially influence product strategies or customer service improvements.
- **Quality of Code and Documentation:** Organization, clarity, and cleanliness of the Python code. Comprehensive documentation that explains the choice of methods, process of implementation, and summary of findings.

Recommended Dataset:

- **Amazon Product Reviews Dataset:** This dataset provides a wide range of product reviews, making it an excellent source for sentiment analysis. It includes reviews across various product categories, offering a diverse set of sentiments for analysis.

This project provides students with a practical application of web content mining techniques, specifically in the area of sentiment analysis, enhancing their skills in Python programming, data processing, and analysis. By focusing on e-commerce reviews, students can directly see how their work might apply to real-world business scenarios, aiding in decision-making processes based on customer feedback.

5. NLP and Text Mining

For this project, students will develop a named entity recognition system to extract specific information like company names and financial metrics from news articles. This implementation should be carried out in Python using natural language processing libraries such as NLTK or spaCy. The grading will focus on the accuracy of entity extraction and the handling of various text formats. The Reuters News dataset is suggested for extracting the necessary text data.

Project Details:

- **Objective:** Implement a named entity recognition system using Python that can identify and extract entities like company names and financial figures from a corpus of news articles. The goal is to parse large volumes of text to retrieve structured information that can be used for further analysis or reporting.
- **Tasks:**
    - **Data Preparation:** Load and preprocess the Reuters News dataset to ensure the text is in a suitable format for NLP operations. This may include tasks like sentence segmentation, tokenization, and part-of-speech tagging, which are critical for effective NER.
    - **NER Model Implementation:** Utilize libraries like NLTK or spaCy to build or fine-tune a named entity recognition model. Students may choose to use pre-trained models available within these libraries or train their own models tailored to the specific entities of interest (e.g., financial metrics and company names).

- **Model Evaluation:** Assess the accuracy of the NER system by comparing the extracted entities against a manually annotated set or using pre-defined metrics like precision, recall, and F1-score. Analyze the performance across different types of articles to evaluate robustness.
- **Extraction and Analysis:** Post-extraction, analyze the frequency and context of the extracted entities. Students should aim to provide insights such as which companies are most frequently mentioned in financial contexts or trends in financial reporting.

Grading Criteria:

- **Accuracy of Entity Extraction:** Precision in correctly identifying and extracting the required entities from the text. This involves not just recognizing the entity but also categorizing it correctly as a company name, financial metric, etc.
- **Handling of Text Formats:** Ability to process and extract information from various text formats within news articles, demonstrating robustness and adaptability of the NER system.
- **Innovativeness and Depth of Analysis:** Creativity in the application of NLP techniques for entity extraction and the depth of analysis on the extracted data. Insightful conclusions about the data trends and anomalies in entity occurrence will be valued.
- **Quality of Code and Documentation:** The clarity, organization, and readability of the Python code. Comprehensive documentation that outlines the system design, libraries used, challenges encountered, and methodologies implemented.

Recommended Dataset:

- **Reuters News Dataset:** This dataset offers a broad range of news articles that are rich in information about companies and financial markets, making it an ideal source for developing and testing a financial-focused NER system.

This project allows students to practically apply advanced NLP techniques in a real-world context, enhancing their skills in data extraction and analysis while providing them with the experience needed to tackle similar challenges in various domains, such as finance, marketing, or public relations.

6. Link Analysis with PageRank

This project involves implementing the PageRank algorithm to rank web pages from a small-scale web crawl. Students should use Python for the implementation, possibly employing the NetworkX library to manage graph operations. Projects will be graded based on the correctness of the PageRank implementation and the analysis of its sensitivity to the damping factor. A custom web crawl dataset, gathered using Python-based tools like Scrapy, would be ideal for this project.

Project Details:

- **Objective:** Use Python to implement the PageRank algorithm to rank web pages based on their link structures. The project will involve creating a graph from a web crawl and applying the PageRank algorithm to determine the relative importance of each page in the dataset.
- **Tasks:**
    - **Web Crawling:** Utilize Python-based tools such as Scrapy to perform a small-scale web crawl to gather data. This data will be used to construct the graph, with nodes representing web pages and edges representing hyperlinks between them.
    - **Graph Construction:** Build a graph representation of the crawled web data using the NetworkX library. This graph will form the basis for applying the PageRank algorithm.
    - **PageRank Implementation:** Implement the PageRank algorithm, ensuring to handle edge cases such as dangling links (pages with no outbound links) and spider traps (cycles within the graph that could mislead the ranking process).
    - **Analysis of Results:** Analyze the sensitivity of the PageRank results to changes in the damping factor and the structure of the web graph. This analysis will help understand the stability and reliability of the PageRank algorithm under different conditions.

Grading Criteria:

- **Correctness of PageRank Implementation:** Accuracy in the implementation of the PageRank algorithm, ensuring it correctly computes page ranks based on the link structure of the graph.

- **Handling of Graph Operations:** Proficiency in using the NetworkX library to construct and manipulate the graph data derived from the web crawl. This includes handling complex graph features and ensuring efficient data processing.
- **Analysis of Algorithm Sensitivity:** Depth of analysis on the PageRank algorithm's sensitivity to the damping factor and the web graph's structure. Insights on how different parameters and graph configurations affect the outcome of the algorithm will be particularly valued.
- **Quality of Code and Documentation:** Clarity and organization of the Python code, use of best practices in coding, and comprehensive documentation that describes the methodology, challenges, and findings of the project.

Recommended Dataset:

- **Custom Web Crawl Dataset:** A dataset specifically crawled for this project using Scrapy or another Python-based crawling tool. This dataset should ideally contain a manageable number of web pages but enough complexity in its link structure to test the PageRank algorithm effectively.

This project is an excellent opportunity for students to apply theoretical concepts of link analysis and graph theory in a practical, hands-on setting. It enhances their understanding of how search engines rank pages and the technical challenges involved in implementing algorithms that can scale to the size of the web.

7. Recommender Systems

Students will build and evaluate a movie recommendation system using collaborative filtering techniques, implemented in Python. The use of libraries such as Pandas for data manipulation and Scikit-Learn or Surprise for building the recommendation system will be expected. The grading will assess the functioning of the recommender system, its accuracy using metrics like precision and recall, and insights into user segmentation. The MovieLens dataset is highly recommended for this project.

Project Details:

- **Objective:** Develop a collaborative filtering movie recommendation system in Python. The system should predict user ratings for movies they have not yet watched and recommend movies based on these predictions.
- **Tasks:**

- **Data Preparation:** Utilize the MovieLens dataset to create a user-item ratings matrix. Students will use Pandas to handle data cleaning, exploration, and transformation tasks, setting up the data for the recommendation model.
- **Model Building:** Implement collaborative filtering algorithms using either Scikit-Learn or the Surprise library. Students may choose to focus on user-based or item-based collaborative filtering techniques, or experiment with matrix factorization methods.
- **Model Evaluation:** Evaluate the recommendation system using metrics such as RMSE (Root Mean Squared Error), precision, recall, and F1-score to assess the accuracy of the predictions. Additionally, analyze the system's ability to handle cold start problems or sparse data issues.
- Insight Generation and Reporting: Analyze the performance of the recommender system and provide insights into user behavior and preferences. Discuss the effectiveness of different filtering approaches and how they influence the recommendations provided to users.

Grading Criteria:

- **Functionality of Recommender System:** The system's ability to accurately predict ratings and make meaningful recommendations. Functionality will be assessed based on the system's design, implementation, and operational efficiency.
- **Accuracy of Model Predictions:** Precision and accuracy in predicting user ratings, evaluated through predefined metrics like precision, recall, and F1-score.
- **Depth of Insights:** The quality of insights derived from the recommendation system regarding user preferences and behavior. Evaluation of how well the system addresses common issues in recommender systems such as scalability, sparsity, and cold start.
- **Quality of Code and Documentation:** The organization, readability, and documentation of the Python code. Documentation should clearly explain the choice of algorithms, the system architecture, and the findings from the evaluation phase.

Recommended Dataset:

- **MovieLens Dataset:** This dataset is a staple in movie recommendation system projects and provides a comprehensive set of user ratings for movies, which is ideal for building and testing collaborative filtering algorithms.

This project provides a comprehensive introduction to the challenges and methodologies involved in building recommender systems. It offers students practical experience with data science and machine learning techniques in Python, preparing them for more complex data analytics projects in the future.

8. Graph Neural Network for Web Mining

In this project, students will use Graph Neural Networks to analyze web data structured as graphs. The primary objective will be to predict properties of web entities (such as webpages or users) or to classify nodes in a social or information network. Students will implement GNN models using Python, leveraging libraries like PyTorch Geometric or DGL (Deep Graph Library), which are well-suited for efficient and scalable GNN architectures.

Project Details:

- **Objective:** Utilize GNNs to model and predict user behavior on social media platforms or to classify webpages based on their content and link structure.
- **Tasks:**
    - **Data Preparation:** Convert raw web data into a graph format where nodes represent entities (e.g., users, webpages) and edges represent interactions/links (e.g., user follows, hyperlinks).
    - **Model Implementation:** Implement a GNN model to learn node embeddings that capture the complexities and dependencies in the web graph. This could involve predicting the category of webpages, the interests of users, or the influence of a webpage based on its links.
    - **Evaluation:** Evaluate the model based on accuracy, precision, recall, and other relevant metrics. Analyze the performance of the model in different scenarios or configurations to understand its effectiveness and limitations.

Grading Criteria:

- **Accuracy of GNN Implementation:** Correctness and efficiency of the Python implementation using GNN frameworks.
- **Innovativeness in Model Application:** Creativity in applying GNN to a web mining problem, such as innovative ways of graph representation or novel use cases.
- **Depth of Analysis:** Quality of insights derived from the model outputs, including interpretation of node embeddings and analysis of model predictions.
- **Quality of Code and Documentation:** Clarity and organization of the code, along with comprehensive documentation explaining the model architecture, data flow, and results.

Recommended Dataset:

- **Wikipedia Graph Dataset:** A graph dataset where nodes represent Wikipedia articles and edges represent the hyperlinks between them. This dataset can be used to classify articles into categories based on the structure and link context or to predict the quality of articles based on their link patterns.

This project not only familiarizes students with advanced techniques in machine learning and web mining but also enhances their skills in Python programming and graph theory applications in real-world scenarios. By implementing and analyzing a GNN model, students can gain valuable insights into the dynamic and interconnected nature of web data.

A. Deliverables:

- Jupyter Notebook or Python script containing the implementation of the chosen project model,
- Documentation report summarizing the final project tasks, methodology, results, and analysis,
- Visualizations or plots illustrating key findings or insights from the experiment,
- Presentation slides summarizing key findings and insights for presentation purposes.

B. Submission Guidelines:

Submit your final project on Moodle. Ensure that all required deliverables are included and organized in a clear and structured manner. If applicable, provide instructions for reproducing your experiments or running your code.